

《并行算法》理论作业1

陳日康 信息与计算科学 22336049

1. 课本习题 9.3 :

给定模式串 $pat[1:m]$ ，失效函数 F 定义为： $F[1] = 0$ 和 $F[j] = j - D[j]$ ($j > 1$)，其中 $D[j]$ 是前缀 $pat[1:j-1]$ 的周期，计算函数 $F[i]$ 的算法如下：

算法 9.18：计算失效函数 $F[i]$ 的顺序算法

输入：模式串 $pat[1:m]$ 。

输出： $F[k]$ ， $1 \leq k \leq m+1$

```
begin
  (1)  $F[1] \leftarrow 0, F[2] \leftarrow 1, k \leftarrow 2, j \leftarrow 1$ 

  (2) while  $k \leq m$  do
    (2.1) if  $pat[k] = pat[j]$  then
       $j \leftarrow j + 1, k \leftarrow k + 1, F[k] \leftarrow j$ 
    end if

    (2.2) if  $pat[k] \neq pat[j]$  then
      (i)  $j \leftarrow F[j]$ 
      (ii) if  $j = 0$  then
         $k \leftarrow k + 1, j \leftarrow 1, F[k] \leftarrow 1$ 
      end if
    end if
  end while
end
```

设 $pat = \text{abcabcbcabcbcc}$ ， $m = 16$ ，试计算 $F[1] \sim F[17]$ ；

证明此算法的正确性和时间复杂度为 $O(m)$ 。

解：

(1). 失效函数值计算过程：

根据算法 9.18，初始化：

```
 $F[1] \leftarrow 0, F[2] \leftarrow 1, k \leftarrow 2, j \leftarrow 1$ 
```

随后依次比较 $pat[k]$ 与 $pat[j]$ ，若相等则 $F[k+1] \leftarrow j+1$ ，否则将 j 回退为 $F[j]$ ，继续比较，直到找到匹配或 $j=0$ 。

依此算法，可得：

k	pat[k]	pat[j]	结果	F[k+1]	j 更新
2	b	a	不相等	1	$j \leftarrow 0 \rightarrow 1$
3	c	a	不相等	1	$j \leftarrow 0 \rightarrow 1$
4	a	a	相等	2	$j \leftarrow 2$
5	b	b	相等	3	$j \leftarrow 3$
6	c	c	相等	4	$j \leftarrow 4$
7	a	a	相等	5	$j \leftarrow 5$
8	b	b	相等	6	$j \leftarrow 6$
9	c	c	相等	7	$j \leftarrow 7$
10	a	a	相等	8	$j \leftarrow 8$
11	b	b	相等	9	$j \leftarrow 9$
12	c	c	相等	10	$j \leftarrow 10$
13	a	a	相等	11	$j \leftarrow 11$
14	b	b	相等	12	$j \leftarrow 12$
15	c	c	相等	13	$j \leftarrow 13$
16	c	a	不相等		j 回退多次，最终 $j \leftarrow 0$ ， $F[17] \leftarrow 1$

最终失效函数为：

```
F[1] = 0, F[2] = 1, F[3] = 1,
F[4] = 2, F[5] = 3, F[6] = 4,
F[7] = 5, F[8] = 6, F[9] = 7,
F[10] = 8, F[11] = 9, F[12] = 10,
F[13] = 11, F[14] = 12, F[15] = 13,
F[16] = 1, F[17] = 1
```

(2). 算法正确性证明：

算法根据模式串的前缀信息，逐步计算每个位置 k 上的失效函数 $F[k]$ ，即 $pat[1..k-1]$ 中最长真前缀与真后缀相等的长度加一。其核心思想是维护当前已知的最长匹配前缀长度 j ，若 $pat[k] = pat[j]$ ，则说明前缀可以继续扩展，设置 $F[k+1] \leftarrow j+1$ ；若不相等，则根据之前已构造好的 $F[j]$ 进行回退，寻找更短的匹配前缀。这种逐步回退等价于遍历有限状态自动机中的失效路径，直到找到匹配或回退至 $j=0$ 。

因此，该算法能够准确地构建每一位置对应的最长匹配前缀长度，并以线性时间建立起失效函数表。失效函数在 KMP 算法的匹配过程中用于主串失配时的快速跳转位置，从而避免重复比较，实现整个模式匹配过程的线性复杂度。

(3). 时间复杂度分析：

在算法构造失效函数的过程中，虽然在 `pat[k] ≠ pat[j]` 失配时可能多次执行回退操作 `j ← F[j]`，但整个过程中，每个字符最多参与一次回退和一次增加，因此操作总次数是线性的。

外层循环中 `k` 从 2 到 `m + 1`，共进行 `m` 次迭代；每当匹配成功时，变量 `j` 增加一次，对应最长前缀长度的增长，最多增加 `m - 1` 次；而当匹配失败时，通过 `j ← F[j]` 回退前缀匹配，每个字符最多经历一次回退，因而总回退次数也不会超过 `m`。

综上，整个构造过程中文本字符的处理次数不超过 `2m` 次，时间复杂度为线性，即算法时间复杂度为： $O(m)$ 。

2. 课本习题 9.9

试修改水平和斜向双并行计算编辑距离的允许 `k`-差别的近似串匹配并行算法使之适用于分布存储的机群并行计算系统。

解：为了将支持水平和斜向双方向并行的、允许 `k` 差别的近似串匹配算法扩展到分布式存储结构下的机群并行计算系统中，必须对其原有的并行策略进行重新设计。该算法原本运行在共享内存环境中，依赖多线程对整个动态规划矩阵的并行访问与更新。在分布式内存系统中，节点间无法直接共享数据，因此首先需要对数据划分方式进行调整，例如将动态规划矩阵按斜线或子块划分，将不同部分分配给不同的计算节点。例如，可采用波前调度（wavefront scheduling）或对角线块划分策略，确保数据依赖顺序被满足。由于矩阵中存在明显的数据相关性，节点之间需通过 MPI 的点对点通信（如 `MPI_Send` 和 `MPI_Recv`）交换边界数据，以维持计算正确性，从而替代原有的共享内存访问机制。

在设计通信机制的同时，还要考虑同步与调度策略，确保依赖关系得到正确维护，不发生阻塞或等待。同时，为了提高整个系统的并行效率，还应根据矩阵结构与计算负载，合理分配任务，尽可能实现负载均衡。通过上述重构，该算法可以在不牺牲匹配精度的前提下，有效利用分布式系统的计算资源，实现大规模文本下的高效近似串匹配。